

L O A O B O C O O G O F O I O O H O O O J O O O O E O O K O O O O D O O O O

2026 江西八校集训联盟第二场

题解讲评

东华理工大学命题组

2026 江西八校集训联盟

2026 年 8 月 30 日

预期难度

由易到难

$L \rightarrow A \rightarrow B \rightarrow C \rightarrow G \rightarrow F \rightarrow I \rightarrow H \rightarrow J \rightarrow E \rightarrow K \rightarrow D$

- 阶段一 (签到): L、A、B、C;
- 阶段二 (基础算法): G、F、I;
- 阶段二 (思维进阶): H、J、E;
- 阶段四 (高级算法、思维能力): K、D。

简要题意

给定作文中的 n 个单词，统计其中有多少个单词恰好等于 `xnn`。

- 按单词依次读入字符串；
- 判断它是否与 `xnn` 完全相同，若相同则答案加一。

复杂度

时间复杂度为 $O(\text{总字符串长度})$ 。

A. Seat numbering

简要题意

赛场共有 n 排、每排 m 个座位。给定第 r 排第 c 列，输出它的编号以及前、后各有多少个座位。

座位按照行优先的顺序编号。第 r 行之前共有 $(r-1)m$ 个座位，因此当前位置的编号为

$$id = (r-1)m + c.$$

- 排在当前位置之前的座位数为 $id - 1$;
- 排在当前位置之后的座位数为 $nm - id$ 。

复杂度

时间复杂度为 $\mathcal{O}(1)$ 。

B. Grounded candy

简要题意

第 i 根绳子长 b_i , 连接高度为 a_i 的钉子与同一颗糖果。求至少剪断多少根绳子, 才能把糖果放到地面。

结论

第 i 根绳子可以保留, 当且仅当

$$b_i \geq a_i.$$

因此答案就是满足 $b_i < a_i$ 的绳子数量。

输入保证初始状态合法, 只用于说明所有绳子一开始都没有断, 不影响上述判断。

复杂度

一次扫描, 时间复杂度为 $\mathcal{O}(n)$ 。

C. Echo tape

简要题意

在一条环形数字磁带上执行 k 轮读数、写回、转向和移动操作，输出最终位置、得分与磁带内容。

每一轮只需要模拟当前位置发生的变化：

- ① 读出当前位置的旧数字 x ，将其加入得分；
- ② 把当前位置改为 $(x + 1) \bmod 10$ ；
- ③ 若 $x = 0$ ，反转方向且本轮不移动；
- ④ 否则沿当前方向移动 x 步，并用模 n 处理环形下标。

C. Echo tape

注意：如果你不知道为什么 WA 请思考利用模 n 处理环形下标时是否正确处理了模 n 后为负数的情况！

复杂度

总轮数满足 $\sum(n + k) \leq 2 \cdot 10^5$ ，逐轮模拟即可，时间复杂度为 $\mathcal{O}(n + k)$ 。

G. Signal overwrite (Easy)

简要题意

每次可以让一座信号塔把自己的值覆写给相邻信号塔。判断能否用任意次操作把序列 A 变为 B 。

- 一次操作只会让一个连续相等段向左或向右扩展，或者让一个长度为 1 的连续段消失。它不能交换两个不同信号段的先后顺序。
- 所以只需要把序列中每个极长连续相等段只保留一个数，称为压缩序列。例如， $[1, 1, 2, 2, 1]$ 的压缩序列是 $[1, 2, 1]$ 。
- 设 A' 和 B' 分别是 A 和 B 的压缩序列。答案为 Yes 当且仅当 B' 是 A' 的子序列。

复杂度

时间复杂度为 $\mathcal{O}(n)$ 。

F. Fixed-Window

简要题意

可不操作，或选择一个长度恰为 L 的区间，将其中频率全部加 k 。求某个频率能够取得的最大权值和。

设 $score_x$ 表示当前操作后，最终频率等于 x 的所有信号权值和。

- 位置 i 加入窗口： $score_{a_i-} = w_i$, $score_{a_i+k+} = w_i$;
- 位置 i 离开窗口时执行相反修改；
- 相邻窗口只改变两个位置，使用滑动窗口维护全部 $score$;
- 大根堆保存 $(score_x, x)$ ，配合懒删除取得当前最大值；
- 注意不进行操作的状态也需要参与答案比较。

值域为 $[-2 \cdot 10^6, 2 \cdot 10^6]$ ，可以直接使用数组，无需离散化。

复杂度

$\mathcal{O}(V + n \log n)$ ，其中 $V = 4 \cdot 10^6 + 1$ ；答案使用 `long long`。

I. Harmonious subarrays

简要题意

若两个序列相差同一个整数平移量，则称它们和谐。每次询问统计区间内与模式串 t 和谐的长度 m 子数组数量。

- 直接枚举每个询问中的所有窗口，再逐项检查，最坏情况下需要 $O(qnm)$ 的时间，无法通过本题的数据范围。
- 关键是消去每个窗口都可能不同的平移量 x 。
- 对于任意相邻两项：

$$b_{i+1} - b_i = (a_{i+1} + x) - (a_i + x) = a_{i+1} - a_i.$$

- 因此整体平移不会改变相邻差分。反过来，如果两个等长序列的所有相邻差分都相同，令 $x = b_1 - a_1$ ，便可沿着序列依次推出每个位置都满足 $a_i + x = b_i$ 。
- 所以，两个等长序列和谐，当且仅当它们的相邻差分序列完全相同。

I. Harmonious subarrays

- ① 对 s, t 分别求相邻差分, 在 D_s 中用 KMP 匹配 D_t ;
- ② 将每个匹配位置记为合法窗口起点, 并对起点数组求前缀和;
- ③ 若询问区间长度 $r - l + 1 < m$, 答案为 0; 否则合法起点范围为 $[l, r - m + 1]$;
- ④ 特判 $m = 1$, 此时每个单点窗口都合法。

复杂度

总时间复杂度为 $\mathcal{O}(n + m + q)$ 。

H. Signal overwrite (Hard)

简要题意

每次可以把一座信号塔的值覆写给距离不超过 k 的另一座信号塔。判断能否把 A 变为 B 。

考虑以下结论：

- ① 若 $A = B$ ，不进行操作即可；
- ② 若 B 出现了 A 中没有的值，必定无解；
- ③ $k = 1$ 时退化为 Easy，判断 B' 是否为 A' 的子序列；
- ④ $k \geq 2$ 时，只需判断 B 中是否存在距离不超过 k 的相等元素对。

H. Signal overwrite (Hard)

前三个结论比较显然，我们来考虑第四个结论的正确性：

- 首先我们倒着理解操作：对于 B 来说，若当前两个距离不超过 k 的位置具有相同值，那么其中一个位置在上一步可以是任意值。因此， B 中的一对相同近邻可以产生一个通配符 $*$ 。
- 当 $k \geq 2$ 时，可以利用一个通配符完成以下两类倒序调整：
- 1. 将通配符移动到相邻位置；
- 2. 交换通配符旁边的两个相邻具体值。
- 在下面的式子中，箭头表示倒序构造过程。第一类调整为

$$(*, x) \longrightarrow (x, *).$$

- 具体地，先把左侧的通配符取成 x ，得到 (x, x) ；由于两个 x 相邻，可以把右侧的 x 变成新的通配符。

H. Signal overwrite (Hard)

- 第二类调整为

$$(*, x, y) \longrightarrow (y, x, *).$$

- 先把通配符取成 y , 得到 (y, x, y) 。首尾两个 y 的距离为 2, 因此可以把右侧的 y 变成新的通配符, 得到 $(y, x, *)$ 。这里用到了距离为 2 的位置, 所以要求 $k \geq 2$ 。
- 反复使用这两类调整, 可以让通配符移动到任意位置, 也可以任意调整其余具体值的顺序。
- 接下来, 不断把尚未变成通配符的相同值聚在一起, 并把其中一个变成新的通配符, 直到每种值至多留下一个具体位置。
- 因为 B 中的每种值都在 A 中出现, 所以可以重新排列这些个具体值, 让它们分别对准 A 中一次对应值的出现; 其余位置都由通配符匹配。于是这个倒序模式包含初始序列 A 。将整串倒序操作反向执行, 就能从 A 得到 B 。

H. Signal overwrite (Hard)

- 因此, $k \geq 2$ 时只需检查 B 中是否存在距离不超过 k 的相等元素即可。
- 如果存在那么就是 Yes, 否则就是 No。

复杂度

记录每个值最近出现的位置, 线性扫描, 时间复杂度为 $\mathcal{O}(n)$ 。

J. Windows at Odds

简要题意

给定 p, q , 求满足“所有长度 p 窗口和为负、所有长度 q 窗口和为正”的最大长度, 并构造一个合法整数序列。

- 弱化版的原题是 IMO 1977 T2
- 数学上比较漂亮的解法参考: IMO 1977 第 2 题探析

J. Windows at Odds

我们考虑一个更加容易理解的构造方法：

令前缀和 $S_i = \sum_{j=1}^i a_j$ 。两类窗口限制分别变为

$$S_{i+p} < S_i, \quad S_i < S_{i+q}.$$

建立有向图，边 $u \rightarrow v$ 表示 $S_v \geq S_u + 1$ ：

- ① 分别加入边 $i+p \rightarrow i$ 与 $i \rightarrow i+q$ ；
- ② 图中存在环，当且仅当所有不等式无法同时满足；
- ③ 二分答案 n （即满足条件的最长长度），每次 *check* 时只要拓扑排序判环即可；
- ④ 最后在最大无环图上跑拓扑最长路得到 S ，再令 $a_i = S_i - S_{i-1}$ 。

J. Windows at Odds

考虑为什么 n 具有二分性：

- 如果长度 n 已经能容纳一个环，那么扩大长度 n 后，原来的环仍然存在。
- 因此存在一个临界值：
 - ① 小于临界值时图是 DAG，对应长度可行；
 - ② 达到临界值后图中有环，对应长度无解。
- 所以 n 具有二分性，可以二分。

复杂度

时间复杂度为 $\mathcal{O}((p+q)\log(p+q))$ 。

J. Windows at Odds

可以通过打表或者数学结论得到最长长度就是：

$$n = p + q - \gcd(p, q) - 1$$

那么就可以把时间复杂度降到 $\mathcal{O}(p + q)$ 。

E. Eternal relay

简要题意

在带正增益的有向图中，从 s 出发恰走 K 条边到达 t ，最大化沿途边权乘积，并输出其前 d 位有效数字。

路径增益是边权乘积。取常用对数后，乘积转化为加法：

$$\log_{10} \left(\prod w_i \right) = \sum \log_{10} w_i.$$

- ① 使用 max-plus 矩阵乘法： $C_{ij} = \max_k (A_{ik} + B_{kj})$;
- ② M^K 表示恰好走 K 条边的最优对数和；
- ③ 矩阵快速幂即可；

E. Eternal relay

考虑如何恢复前导数字：若最优路径乘积的对数为 S ，令

$$f = S - \lfloor S \rfloor,$$

则其科学计数法尾数为 10^f ，前 d 位为

$$\left\lfloor 10^{f+d-1} \right\rfloor.$$

代码实现上需要注意的问题：

- 使用 long double 保存对数；
- 不可达状态设的负无穷一定要够小；

E. Eternal relay

由于题目的精度限制，上述写法已经足够通过本题。不过还可以考虑另一种对精度更友好的思路：

- 使用科学计数法保存有向边权为： $x = c \times 10^e, 1 \leq c < 10$.
- 并进行 max-times 矩阵快速幂。最终的尾数 c 可以直接用于提取前 d 位。
- 这种做法和对数做法保存的是等价信息，但实现中需要同时维护尾数、指数和不可达标记，可能会略显繁琐。

复杂度

两种实现方式的时间复杂度均为 $\mathcal{O}(n^3 \log K)$ 。

K. Saturation point

简要题意

对每个非空区间，计算 a 的区间最小值乘以 b 的最大子集异或和，求所有区间价值之和。

先考虑暴力做法：

- 固定左端点 l ，向右扩展区间，并维护 $b_l, b_{l+1}, \dots, b_r$ 的异或线性基和 $a_l, a_{l+1}, \dots, a_r$ 的最小值，最后求和即可；
- 这样做的时间复杂度是： $\mathcal{O}(n^2 m)$ ，无法通过。

考虑向右拓展区间时线性基有以下性质：

- 线性基秩达到 m 后，可以表示所有 m 位整数；
- 此时最大子集异或恒为 $2^m - 1$ ；

K. Saturation point

- 题目保证 b_i 独立均匀随机，因此线性基通常只需 $m + \mathcal{O}(1)$ 个元素便会满秩；
- 所以拓展右端点时可以按线性基是否满秩，分两类计算答案；
- 对于满秩前直接暴力计算贡献，这部分的时间复杂度为 $\mathcal{O}(nm^2)$
- 对于满秩后我们整体计算一次：定义

$$dp_l = \sum_{r=l}^n \min_{l \leq i \leq r} a_i.$$

- 设 nxt_l 是 l 右侧第一个满足 $a_{\text{nxt}_l} < a_l$ 的位置；如果不存在，就令 $\text{nxt}_l = n + 1$ 。
- 所有 nxt_l 可以从右到左用单调栈在 $\mathcal{O}(n)$ 时间内求出。

K. Saturation point

- 那么对于 dp_l 则有递推式:

$$dp_l = a_l(nxt_l - l) + dp_{nxt_l}.$$

- 若第一次满秩的右端点为 R , 后续贡献可以一次计算为

$$(2^m - 1) \sum_{r=R+1}^n \min(a_l, \dots, a_r).$$

- 即:

$$(2^m - 1) \left(dp_l - \sum_{r=l}^R \min_{l \leq i \leq r} a_i \right)$$

- $\sum_{r=l}^R \min_{l \leq i \leq r} a_i$ 在满秩前暴力算即可。

K. Saturation point

复杂度

随机模型下时间复杂度为 $\mathcal{O}(nm^2)$ 。

D. Sqrt range add

简要题意

维护一个非负整数序列，支持区间加，以及查询区间内 $\sum \lfloor \sqrt{a_i} \rfloor$ 。

令

$$val_i = \lfloor \sqrt{a_i} \rfloor.$$

val_i 只会在 a_i 到达下一个完全平方数 $(val_i + 1)^2$ 时增加。

相邻两个完全平方数之间的距离为：

$$(v+1)^2 - v^2 = 2v + 1.$$

题目保证最终所有元素都不超过 $V = 3 \times 10^5$ ，所以始终有 $v \leq 547$ ，相邻两次事件的距离至多为

$$2 \times 547 + 1 = 1095.$$

这个较小的事件间距是本题的关键。

D. Sqrt range add

- 考虑对序列分块:
- 对于每个位置 i , 维护:
 - ① a_i : 不包含所在块懒标记的基础值;
 - ② val_i : 当前真实值的平方根下取整。
- 对于每个块 b , 维护:
 - ① tag : 整块加法的懒标记;
 - ② sum : 块内所有 val_i 的和。
 - ③ $events_t$: 当块内 tag 为 t 时会使得 val_i 增加的所有的 i (即可能影响答案的位置)。
- 若当前位置的 val_i 为 v , 那么下一次变化对应的 tag 的值为 $(v+1)^2 - a_i$;

D. Sqrt range add

- 修改时:
- 对于完整块的加法我们可以利用 *events* 的信息去修改块内的答案和维护新的 *events*;
- 相邻两次事件的距离至多为 1095, 每块使用 `array<vector<int>, 1105>` *events* 作为循环桶; 桶长 1105 严格大于最大事件间隔。
- 因此原地遍历当前桶时, 下一次事件不会插回当前桶, 避免向正在遍历的 *vector* 插入元素导致迭代器失效;
- 并且由于所有加法非负且最终值有界, 每个位置触发事件的总次数受到值域约束: 均摊下来时间复杂度为 $\mathcal{O}(n\sqrt{V})$;
- 两边的边界块直接下放懒标记后逐项修改并重建整个块即可。

D. Sqrt range add

- 查询时:
- 两个边界块逐项计算;
- 中间完整块直接累加块内维护的平方根和 sum 。

取块长 $B \approx 640$

复杂度

总时间复杂度为:

$$\mathcal{O}\left(qB + \frac{qn}{B} + \frac{nV}{B} + n\sqrt{V}\right)$$

D. Sqrt range add

由于要照顾到 *Python* 选手，所以时限开的比较大，可能会让一些小常数

$$\mathcal{O}\left(q \log n + n\sqrt{V} \log n\right)$$

的代码通过，非常抱歉。

Thanks!

感谢大家参加本场比赛